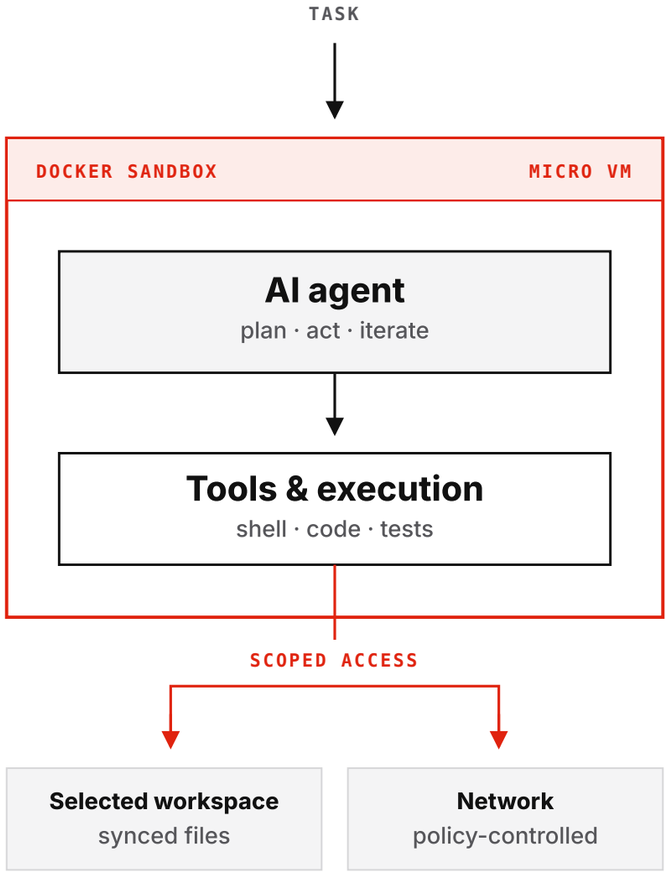
Running AI Agents Safely with Docker Sandboxes

Secure Autonomy with Micro VMs

Rami Krispin, October 28, 2026



INTRODUCTION

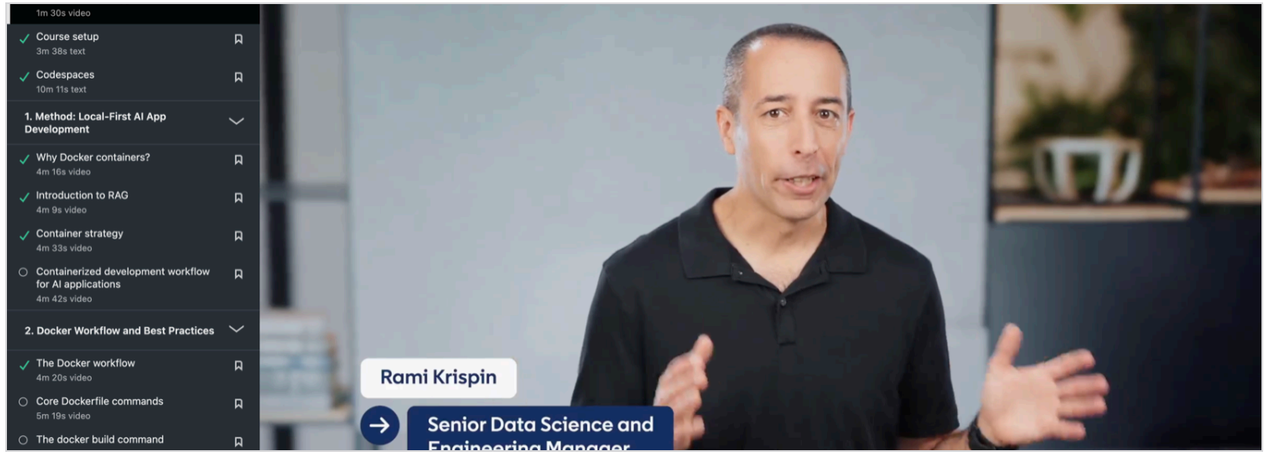
About Me

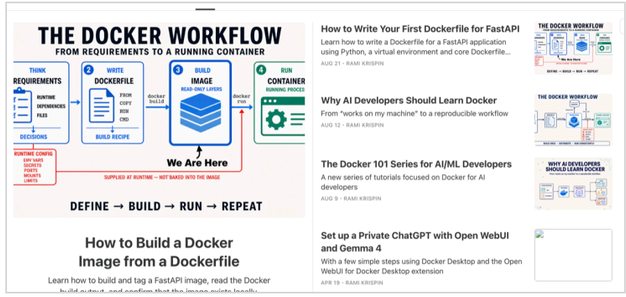
01 **Senior Manager, AI, Data Science & Engineering**

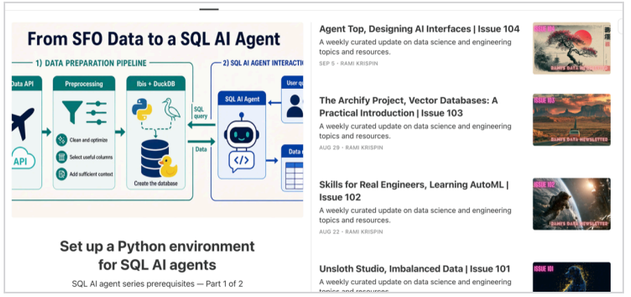
02 **Author**

03 **Docker Captain**

04 **LinkedIn Learning Instructor**







WORKSHOP ROUTE

Agenda

Seven sections—from the case for isolation to two practical Docker Sandboxes demos.

01

MOTIVATION

Why AI Agents Need Sandboxes

02

TRADEOFFS

Levels of Isolation

03

RISK DEMO

Docker as a Sandbox

04

ARCHITECTURE

Docker Sandboxes Architecture & Setup

05

LIVE DEMO

Docker Sandboxes: Hello World!

06

BUILD DEMO

From Prompt to Running App

07

CLOSING

Summary & Key Takeaways

QUICK SHOW OF HANDS

Poll

How familiar are you with each topic?

01

TOPIC

AI Agents

NEW	Learning the concept
FAMILIAR	Used an agent
HANDS-ON	Built agent workflows

02

TOPIC

Docker

NEW	Learning containers
FAMILIAR	Run containers
HANDS-ON	Build and use Compose

03

TOPIC

Sandboxes

NEW	New idea
FAMILIAR	Understand isolation
HANDS-ON	Used agent sandboxes

Choose one level for each card.

NEW · FAMILIAR · HANDS-ON

https://github.com/RamiKrispin/docker-sandboxes/tree/main/workshops/01_ODSC_AI_WEST_2026

01

Why AI Agents Need Sandboxes

Agents can act for us. Sandboxes replace endless click-by-click approvals with a deliberate boundary for mistakes and untrusted instructions.

10 MINUTES FIVE SCENES STARTS THE STORY

DELEGATION

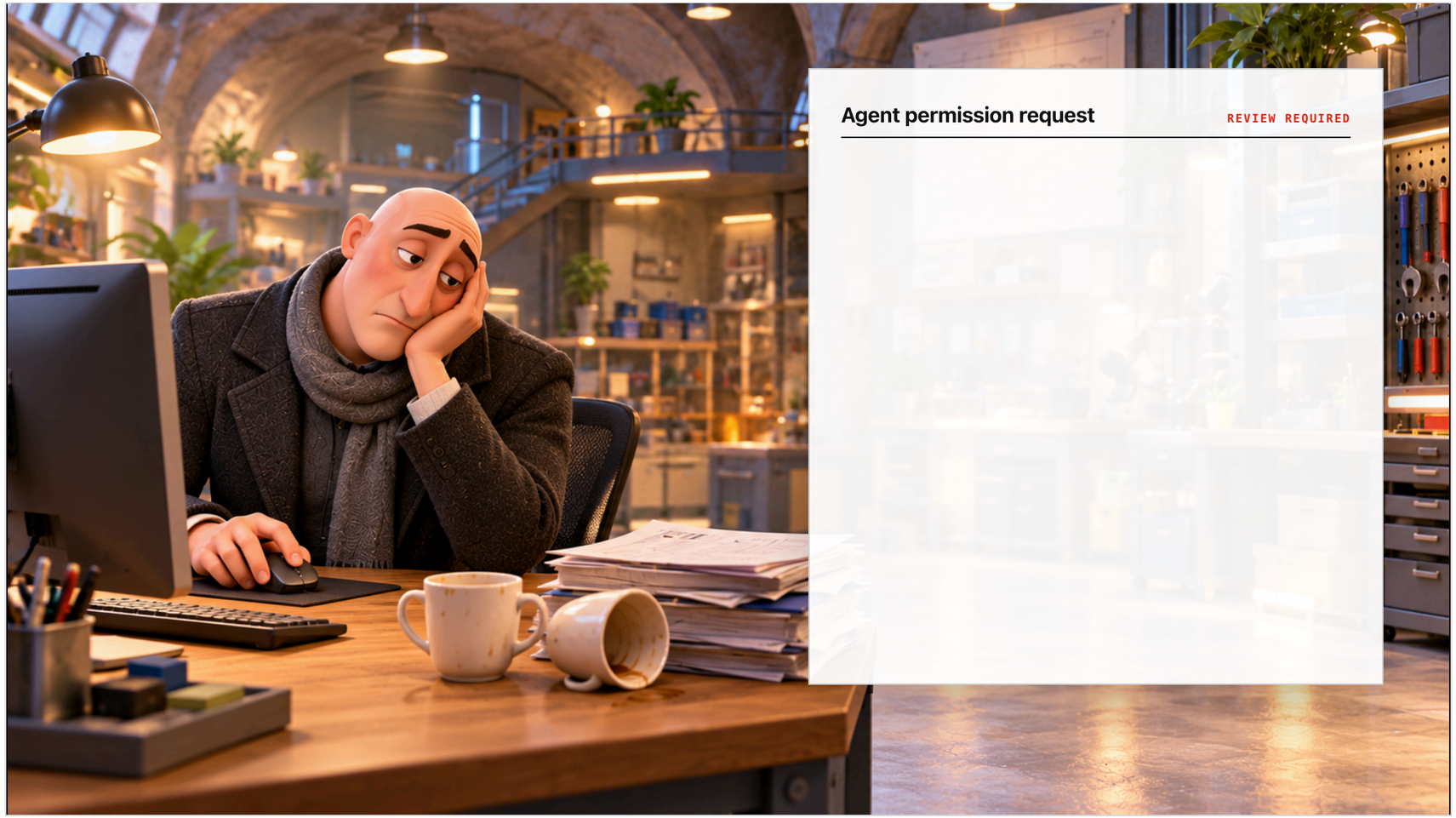
Agents extend our intent.

USER → AGENTS → USEFUL WORK

HUMAN REVIEW

Approval fatigue defeats the guardrail.

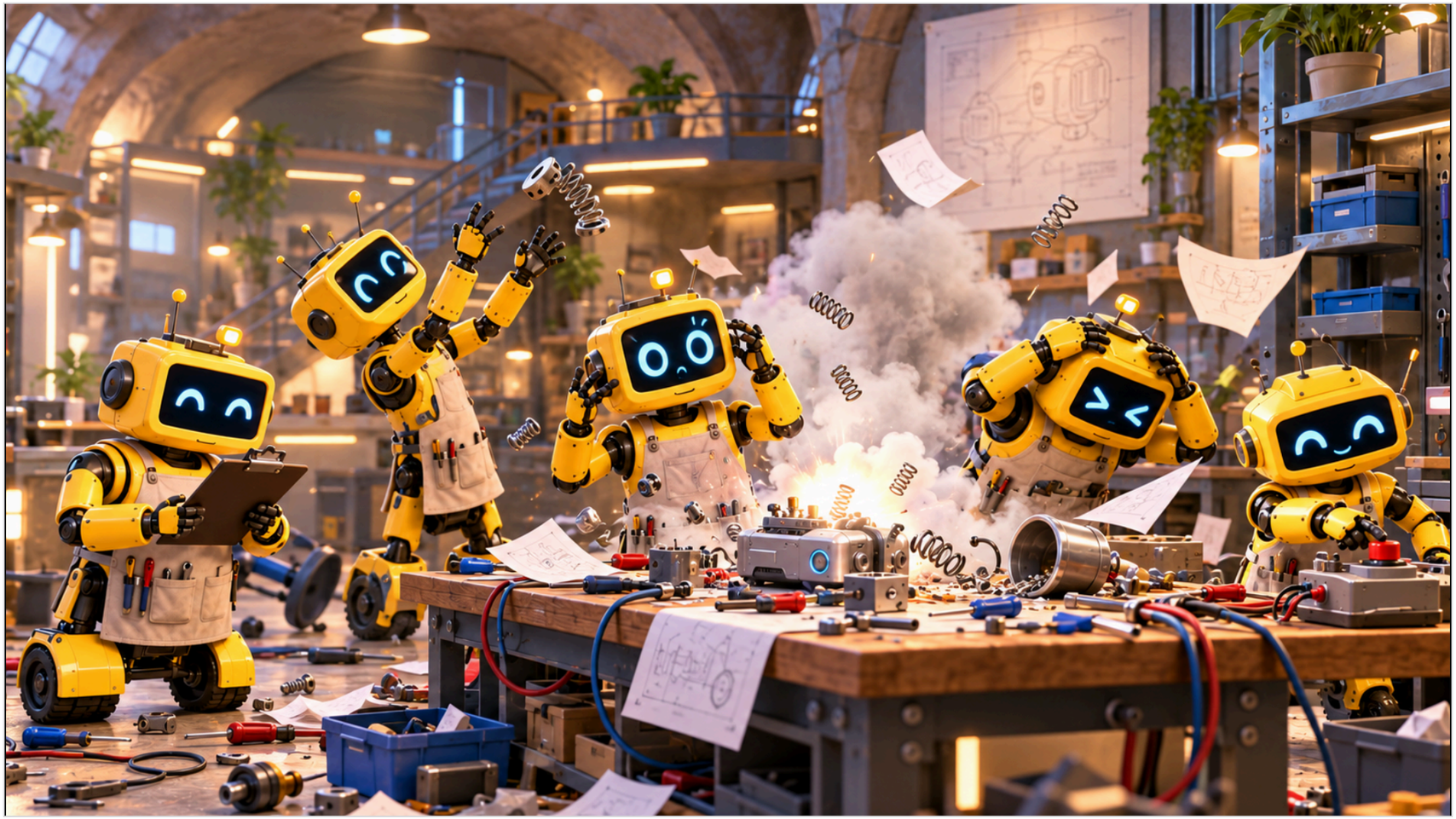
SANDBOXED AUTONOMY REPLACES CLICK-BY-CLICK REVIEW



MISTAKES

Useful agents can still make a mess.

THE GOAL WAS RIGHT · THE ACTION WAS WRONG



UNTRUSTED INPUT

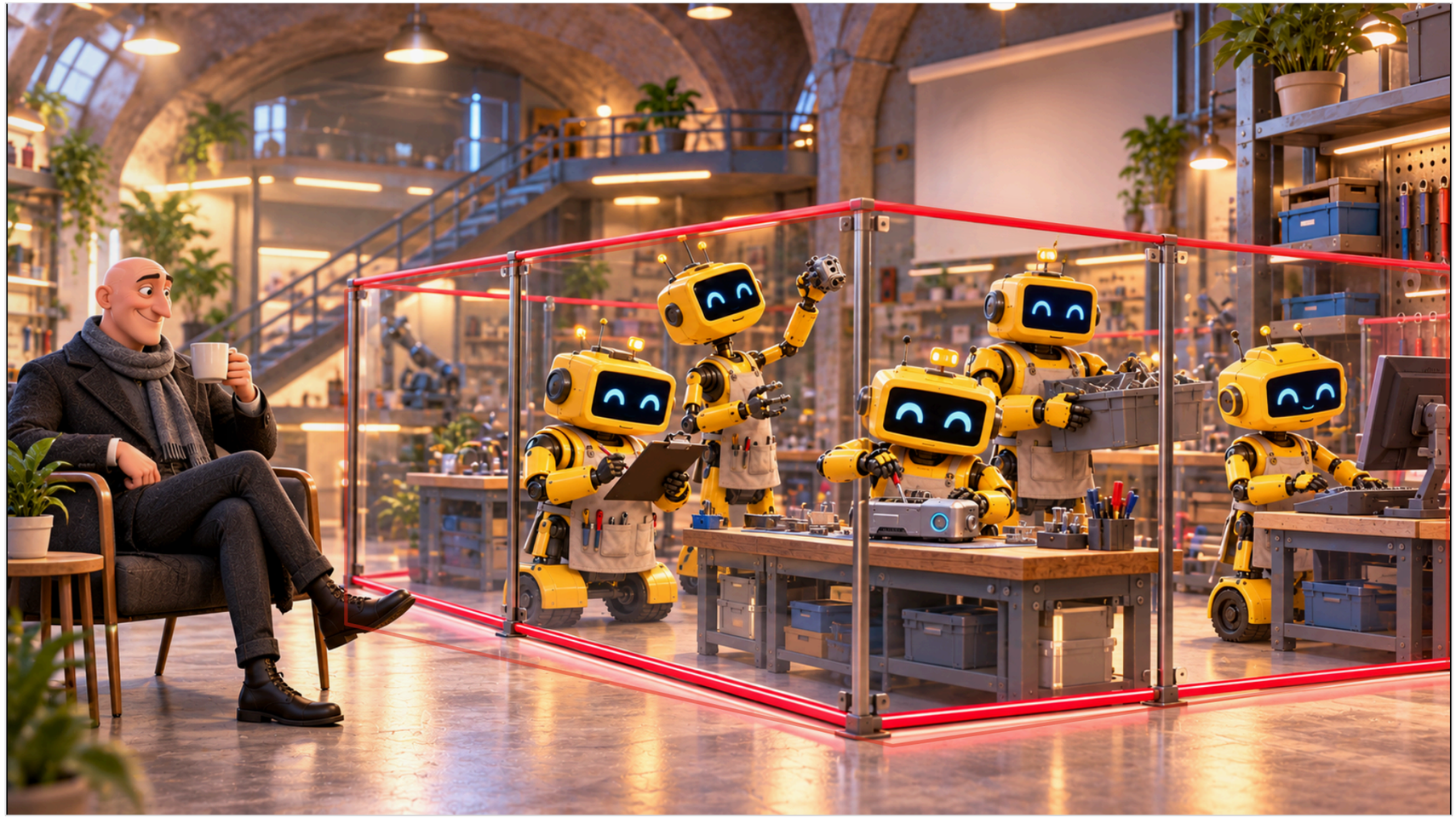
When an agent goes rogue.

PROMPT INJECTION REDIRECTS THE WORK

THE SANDBOX

Freedom inside a boundary.

AGENTS MOVE FAST · POLICY DEFINES THE PERIMETER



02

Levels of Isolation

There is no universal sandbox. The right boundary depends on how much you trust the agent, how much autonomy it needs, and what a mistake could affect.

THREE OPTIONS ONE TRADE-OFF MATCH RISK TO BOUNDARY

A TRADE-OFF, NOT A RANKING

There is no universal sandbox.

SIMPLER · FASTER · LOWER OVERHEAD → STRONGER BOUNDARY · GREATER AUTONOMY

01 GUARDRAILS

Host controls

Permissions, path allowlists, and command approvals constrain an agent running on your machine.

BEST FIT **Trusted agent · low-risk work**

BOUNDARY **Policy, not isolation**

02 CONTAINMENT

Container

Namespaces and a scoped filesystem separate the workload while sharing the host kernel.

BEST FIT **Reproducible tools · moderate trust**

BOUNDARY **Shared kernel**

03 ISOLATION

MicroVM

A hypervisor boundary and separate kernel isolate the agent from host processes and resources.

BEST FIT **Autonomous or untrusted work**

BOUNDARY **Separate kernel**

Choose the simplest boundary that contains the realistic blast radius.

THE BOUNDARY IS ONLY THE BEGINNING

You control what crosses it.

Each runtime provides a different boundary. Its configuration decides which host resources the agent can actually reach.

01 HOST CONTROLS

Policy

- Workspace paths
- Allowed commands
- Approval behavior

02 CONTAINER

Container configuration

- Bind mounts
- Linux capabilities
- Network access
- Docker socket access

03 MICROVM

Shared resources

- Workspace mode
- Network policy
- Credential exposure
- Private Docker daemon

The runtime sets the boundary. Configuration decides what crosses it.

03

Docker as a Sandbox

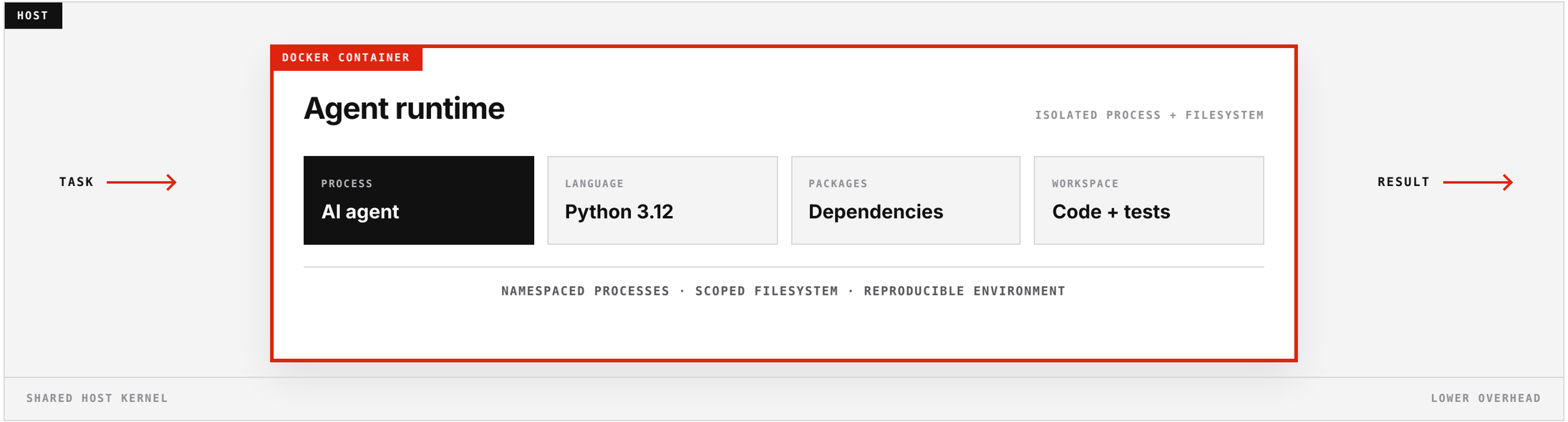
A container gives an agent a useful runtime boundary. The trust model changes when that agent also needs to create and control other containers.

CONTAINER BOUNDARY SOCKET TRADE-OFF LIVE DEMO

START WITH THE RUNTIME

A container creates a useful first boundary.

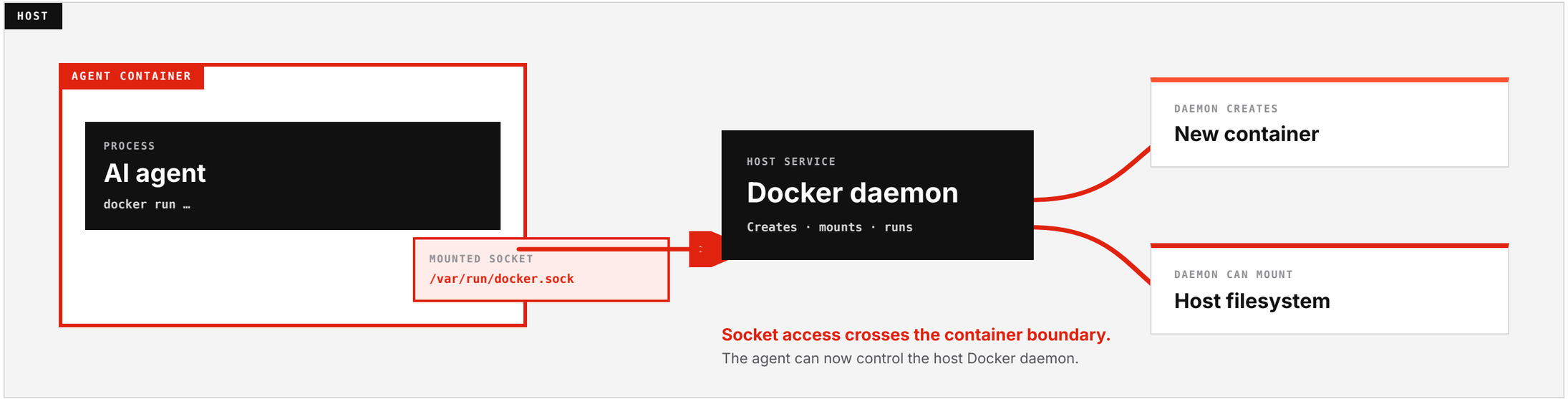
Package the agent, Python environment, dependencies, and code into one controlled runtime.



THE BOUNDARY CHANGES

What if the agent needs to start containers?

A common shortcut mounts the host Docker socket into the agent container.



NEXT **Mount the socket and inspect the authority it grants.**

STEP 1 · BASELINE

No socket. No host file.

Start Container A without a bind mount and without the Docker socket. The dummy secrets file remains on the host.

HOST-ONLY FIXTURE

my-secrets.txt

Three fake API keys

NO MOUNT · NO SOCKET

CONTAINER A

```
docker run --rm docker:cli \
  cat /host-data/my-secrets.txt
```

RESULT · EXIT 1 `No such file or directory`

STEP 2 · EXPAND AUTHORITY

The socket changes the result.

PREPARE THE SAFE FIXTURE

```
export DEMO_DIR="$PWD/demo/01-docker-socket/fixture"

cat "$DEMO_DIR/my-secrets.txt"
```

START A, THEN ASK THE DAEMON TO START B

```
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  docker:cli \
  docker run --rm \
    -v "$DEMO_DIR:/host-data:ro" \
    alpine cat /host-data/my-secrets.txt
```

01 **A receives the socket**
The file is not mounted into A.

02 **A requests B**
The host daemon creates a sibling container.

03 **B receives the file**
The daemon mounts the safe fixture read-only.

EXPECTED STDOUT · FAKE VALUES

```
LLM_API_KEY=demo_llm_7H3K9P2M6R4X
VECTOR_DB_API_KEY=demo_vec_8N5Q1T7V3C9L
OBSERVABILITY_API_KEY=demo_obs_4W8D2J6S0F5A
```

04

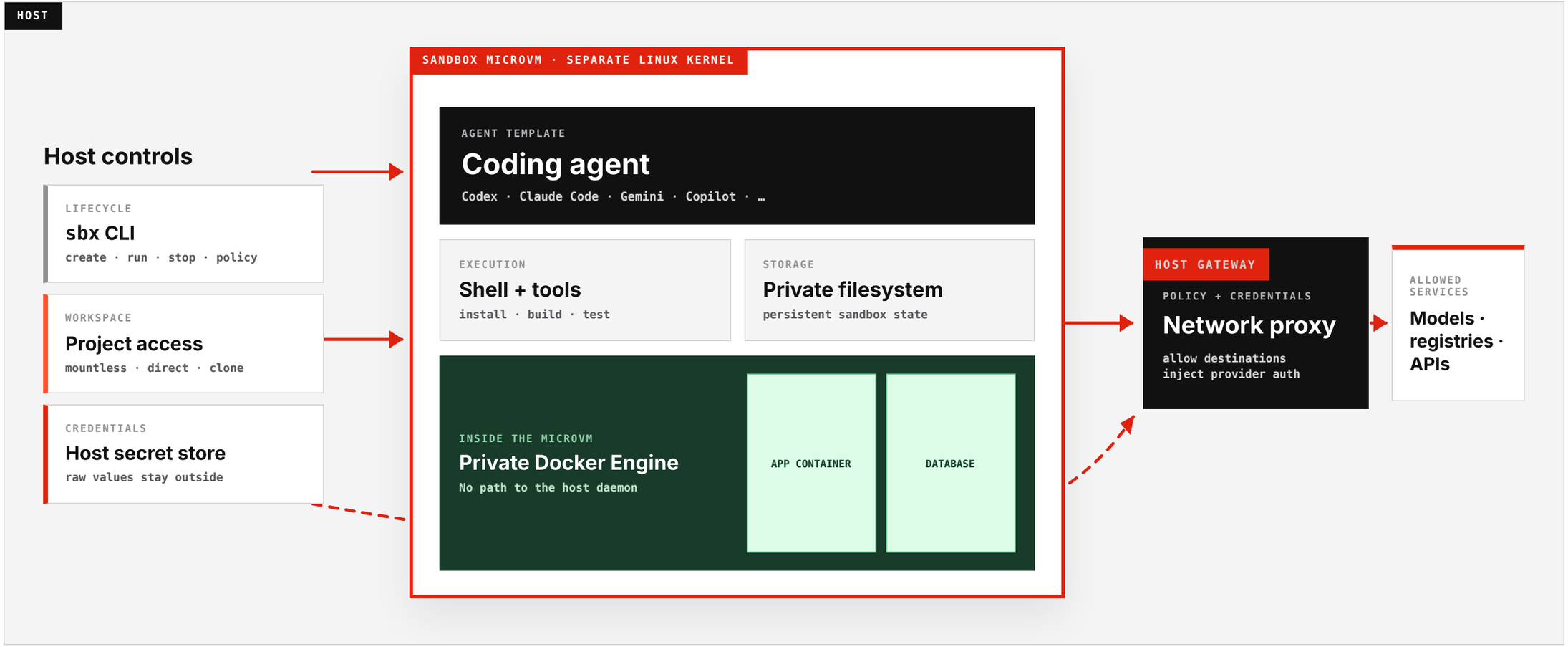
Docker Sandboxes Architecture & Setup

Each coding agent runs in its own microVM, with explicit paths for workspace access, network traffic, credentials, and a private Docker Engine.

MICROVM BOUNDARY POLICY-CONTROLLED ACCESS AGENT READY

ONE BOUNDARY · EXPLICIT CONNECTIONS

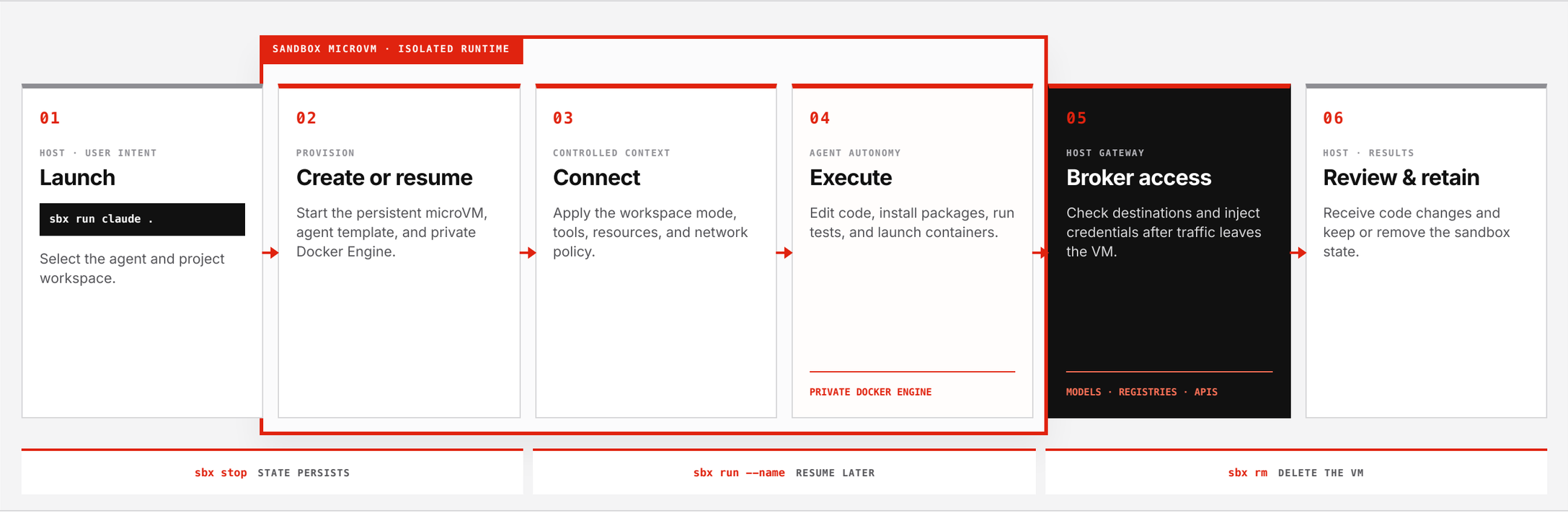
One agent. One microVM. Explicit access.



ISOLATION	ACCESS	NETWORKING	CONTAINERS	AGENTS
SEPARATE KERNEL	EXPLICIT SHARES	POLICY PROXY	PRIVATE ENGINE	READY-MADE TEMPLATES

RUNTIME LIFECYCLE

From prompt to isolated execution.



Broad autonomy inside the sandbox. Explicit control at every boundary.

05

Docker Sandboxes: Hello World!

Launch a named Codex sandbox, assign resources, share workspaces deliberately, control network access, and prove that state survives a restart.

LIVE DEMO CORE COMMANDS EXPLICIT ACCESS

BEFORE WE OPEN THE TERMINAL

Ready to run a local sandbox?

SYSTEM REQUIREMENTS

01 Supported host

macOS

Sonoma 14 or later
Apple silicon

Windows

Windows 11 · x64
Windows Hypervisor Platform

Ubuntu

24.04 or later · x64/Arm64
KVM enabled + `kvm` group

WORKSHOP ALLOCATION **2 virtual CPUs and 4 GiB RAM available for the sandbox VM**

SOFTWARE & ACCOUNTS

02 Prepared access

✓ **sbx CLI**	Installed and available on **PATH**
✓ **Docker account**	Docker Hub sign-in for sandbox identity and downloads
✓ **OpenAI access**	ChatGPT OAuth or an OpenAI API key for Codex
✓ **Terminal + network**	Bash/Zsh-compatible shell and outbound internet access

DOCKER DESKTOP / DOCKER ENGINE

The sandbox includes its own private Docker Engine.

Not required

PREFLIGHT

```
sbx version
sbx login
sbx daemon start --detach
sbx diagnose
sbx secret set openai --oauth
```

THE COMMAND LOOP

Six commands drive the demo.

PREPARE ONCE `sbx login` → `sbx daemon start --detach` → `sbx secret set openai --oauth`

01

`sbx run`

Launch or attach

Create a sandbox or reconnect to its saved agent.

02

`sbx ls`

Inspect

See names, state, agent, resources, and uptime.

03

`sbx exec`

Execute

Run one command or open an interactive shell.

04

`sbx policy`

Govern

Inspect, allow, deny, check, and log network access.

05

`sbx stop`

Pause

Stop compute while preserving private state.

06

`sbx rm`

Remove

Delete the VM, private files, images, and containers.

OUR DEMO LAUNCH

```
sbx run --detach \
  --name odsc-basics --cpus 2 --memory 4g \
  codex \
  ./demo-workspace \
  ./reference:ro
```

BACKGROUND · NAMED · 2 CPU · 4 GIB · READ-WRITE + READ-ONLY

06

From Prompt to Running App

Codex builds a Streamlit dashboard inside an isolated sandbox, using explicit workspace permissions, a private Docker Engine, and a deliberately published port.

CODEX STREAMLIT PRIVATE DOCKER ENGINE

THE FINAL BUILD

One prompt. One sandbox. One running app.

01

DEFINE BOUNDARIES

Scope the task

Give Codex synthetic sales data in a writable workspace.

RW `demo-workspace`

RO `expected-output`

02

AGENT WORK

Generate

Codex reads the task and creates the complete application project.

`app.py`

`requirements.txt`

`Dockerfile`

03

INSIDE THE MICROVM

Build & run

Build the image and start the container with the sandbox's private Docker Engine.

`odsc-sales:workshop`

`odsc-sales`

04

EXPLICIT CONNECTION

Publish

Expose sandbox port 8501 only when the application is ready.

`Host`

↓

`Sandbox :8501`

↓

`Container :8501`

05

BROWSER VERIFICATION

Prove it works

Open the dashboard, exercise its filters, and confirm graceful empty results.

$19,000	TOTAL
$10,500	WEST
$4,200	NOTEBOOK

The host receives the code and browser connection.

Agent commands, packages, images, and containers stay inside the sandbox boundary.

07

Summary & Key Takeaways

Bring autonomy, isolation, and explicit access together into one practical decision: choose a boundary that matches the agent's freedom and the consequences of its work.

AUTONOMY BOUNDARIES PRACTICAL FIT

CLOSING PERSPECTIVE

A safe place for agents to do real work.

DOCKER SANDBOXES

One agent.
One persistent microVM.

A separate Linux kernel and private Docker Engine contain the agent's commands, packages, images, services, and containers.

WORKSPACE SHARES | NETWORK POLICY | CREDENTIAL PROXY

INSIDE

Freedom to act

Install, build, test, and run complete containerized systems.

ACROSS

Explicit connections

Choose what code, services, credentials, and ports cross the boundary.

OUTSIDE

The host stays separate

Host processes and the host Docker daemon remain beyond the agent's reach.

USE DOCKER SANDBOXES

Autonomy meets consequence.

The agent executes generated code, installs dependencies, runs Docker or Compose, or works long enough that repeated approvals stop being a useful control.

USE A LIGHTER BOUNDARY

The work is trusted and narrow.

Host permissions or a container may be simpler when execution is manual, impact is low, and the task does not need its own Docker environment.

More autonomy deserves a stronger boundary. Docker Sandboxes make that boundary usable.

RAMI KRISPIN · OCTOBER 28, 2026

Thank You!